

Stok ve Sipariş Yönetimi

C# temelleri, OOP, collections, hata yönetimi, LINQ ve async/await konularını tek bir console projesinde birleştir.

Yaklaşık	Çalışma biçimi	Çıktı
180–240 dk	6 küçük aşama	Çalışan mini proje

Bugünün kuralı

Projeyi tek seferde yazma. Her aşamayı çalıştır, commit atılabilir küçük bir noktaya getir, sonra sıradaki aşamaya geç.

Tamamlanma ölçütü

- Menü tekrar çalışıyor ve güvenli giriş alıyor.
- Ürün ekleme, listeleme ve arama çalışıyor.
- LINQ raporu doğru sonuç üretiyor.
- Asenkron sipariş hazırlama tamamlanıyor.
- En az beş test senaryosu uygulanıyor.

1. Proje haritası

Mini proje bir mağazanın ürünlerini bellekte yönetecek. Veritabanı henüz yok; veriler program açık olduğu sürece List<Product> içinde tutulacak.

Aşama	Kazanım
1 · Model	Class, property, constructor
2 · Menü	while, switch, metotlar
3 · Ürün ekleme	TryParse, doğrulama, throw/catch
4 · Listeleme/arama	List, foreach, FirstOrDefault
5 · Rapor	Where, OrderBy, Average, GroupBy
6 · Sipariş	Task, await, Task.WhenAll

Dosya yapısı

```
Gun7-MiniProje/  
├─ Program.cs  
├─ Product.cs  
└─ Gun7-MiniProje.csproj
```

Önce çalışan en küçük parça

İlk hedef bütün proje değil: yalnızca bir Product oluşturup bilgilerini ekrana yazdırmak.

2. Aşama 1 — Product modeli

Küçük çalışan örnek: class'ın nasıl kullanılacağını önce gör.

```
Product ornek = new Product("Klavye", "Çevre Birimi", 850, 5);
Console.WriteLine($"{ornek.Name} - {ornek.Price} TL");
```

Product.cs iskeleti

```
public class Product
{
    public string Name { get; }
    public string Category { get; }
    public decimal Price { get; private set; }
    public int Stock { get; private set; }

    public Product(
        string name,
        string category,
        decimal price,
        int stock)
    {
        // Parametreleri property'lere ata.
    }

    public void StockDecrease(int quantity)
    {
        // quantity pozitif mi?
        // Yeterli stok var mı?
        // Geçersizse exception fırlat.
        // Geçerliyse stoktan düş.
    }
}
```

Neden private set?

Fiyat ve stok dışarıdan rastgele değiştirilemesin. Değişiklikler class'ın kontrollü metotları üzerinden yapılsın.

Aşama 1 görevi

- Constructor atamalarını tamamla.
- Price sıfır veya daha küçükse ArgumentException fırlat.
- Stock negatifse ArgumentException fırlat.
- StockDecrease metodunda quantity ve yeterli stok kontrollerini yaz.

3. Aşama 2 — Tekrarlanan menü

Menü, kullanıcı çıkışı seçene kadar tekrar gösterilir. Önce yalnızca seçim alma kısmını çalıştır.

Küçük çalışan örnek

```
bool devam = true;

while (devam)
{
    Console.WriteLine("1 - Ürünleri listele");
    Console.WriteLine("0 - Çıkış");
    Console.Write("Seçiminiz: ");

    string secim = Console.ReadLine() ?? "";

    switch (secim)
    {
        case "1":
            Console.WriteLine("Listeleme seçildi.");
            break;
        case "0":
            devam = false;
            break;
        default:
            Console.WriteLine("Geçersiz seçim.");
            break;
    }
}
```

Tam menü iskeleti

```
1 - Ürün ekle
2 - Ürünleri listele
3 - Ürün ara
4 - LINQ raporu
5 - Sipariş hazırla
0 - Çıkış
```

Görev

Menüyü while + switch ile kur. Şimdilik seçeneklerin içinde yalnızca hangi işlemin seçildiğini yazdır. Böylece iskeletin çalıştığını doğrula.

4. Aşama 3 — Güvenli ürün ekleme

Kullanıcı girdileri doğrudan güvenilir değildir. Fiyat ve stok için TryParse kullan; iş kurallarını Product constructor'ına bırak.

Türkçe adımlar

- Ad ve kategori oku; boşsa işlemi durdur.
- Fiyatı decimal.TryParse ile doğrula.
- Stoku int.TryParse ile doğrula.
- Product oluşturup listeye ekle.
- Constructor exception atarsa catch ile mesajı göster.

İskelet

```
static void AddProduct(List<Product> products)
{
    Console.Write("Ürün adı: ");
    string name = Console.ReadLine() ?? "";

    Console.Write("Kategori: ");
    string category = Console.ReadLine() ?? "";

    Console.Write("Fiyat: ");
    bool priceOk = decimal.TryParse(
        Console.ReadLine(), out decimal price);

    Console.Write("Stok: ");
    bool stockOk = int.TryParse(
        Console.ReadLine(), out int stock);

    // Boş alan ve TryParse sonuçlarını kontrol et.
    // try içinde Product oluştur ve products.Add kullan.
    // catch içinde ex.Message yazdır.
}
```

Test et

Geçerli ürün, harfli fiyat, negatif fiyat ve negatif stok olmak üzere dört ayrı giriş dene.

5. Aşama 4 — Listeleme ve arama

Listeleme foreach; arama ise FirstOrDefault ile yapılacaktır.

Küçük çalışan arama

```
Product? bulunan = products.FirstOrDefault(  
    urun => urun.Name == "Klavve");  
  
if (bulunan is null)  
    Console.WriteLine("Ürün bulunamadı.");  
else  
    Console.WriteLine($"Bulundu: {bulunan.Name}");
```

Neden Product?

Eşleşme bulunamazsa FirstOrDefault, class için null döndürür. Product?, değişkenin Product veya null olabileceğini açıkça belirtir.

Görev iskeleti

```
static void ListProducts(List<Product> products)  
{  
    // Liste boşsa bilgi ver ve return et.  
    // foreach ile ad, kategori, fiyat ve stok yazdır.  
}  
  
static void FindProduct(List<Product> products)  
{  
    Console.Write("Aranacak ürün: ");  
    string search = Console.ReadLine() ?? "";  
  
    Product? bulunan = products.FirstOrDefault(  
        urun => /* büyük-küçük harfi önemsemeyen koşul */;  
  
    // null kontrolü yap ve sonucu yazdır.  
}
```

İpucu

```
string.Equals(birinci, ikinci, StringComparison.OrdinalIgnoreCase)
```

6. Aşama 5 — LINQ raporu

Rapor ekranında koleksiyon üzerinde sorgular üretilecek. Her sorguyu ayrı değişkende kur; önce doğruluğu, sonra kısaltmayı düşün.

Rapor hedefleri

- Stokta olan ürünleri seç ve fiyata göre pahalıdan ucuza sırala.
- Ortalama fiyatı hesapla.
- En pahalı ürünü bul.
- Kategoriye göre grupla ve her grubun ürün sayısını yazdır.

İskelet

```
static void ShowReport(List<Product> products)
{
    if (products.Count == 0)
    {
        Console.WriteLine("Rapor için ürün yok.");
        return;
    }

    List<Product> inStock = products
        .Where(urun => /* stok koşulu */)
        .OrderByDescending(urun => /* fiyat */)
        .ToList();

    decimal averagePrice = products.Average(
        urun => /* fiyat */);

    Product mostExpensive = products
        .OrderByDescending(urun => /* fiyat */)
        .First();

    var groups = products.GroupBy(
        urun => /* kategori */);

    // Sonuçları anlamlı etiketlerle yazdır.
}
```

Boş liste kontrolü

Average ve First boş koleksiyonda exception oluşturabilir. Bu nedenle raporun başındaki kontrol önemlidir.

7. Aşama 6 — Asenkron sipariş

Stok düşme ve hazırlama süresini tek akışta birleştir. Tam çözüm öğrenci kodu görülmeden verilmez.

İskelet

```
static async Task<string> PrepareOrderAsync(
    Product product,
    int quantity,
    int duration)
{
    // Başlangıç mesajı yaz.
    // product.StockDecrease(quantity) çağır.
    // duration kadar Task.Delay ile bekle.
    // Tamamlanma mesajı döndür.
}

static async Task RunOrdersAsync(List<Product> products)
{
    // Gerekli ürünleri FirstOrDefault ile bul.
    // null kontrolü yap.

    try
    {
        Task<string> order1 = /* çağrı */;
        Task<string> order2 = /* çağrı */;

        string[] results = await /* birlikte bekle */;

        foreach (string result in results)
        {
            /* yazdır */
        }
    }
    catch (Exception ex)
    {
        /* anlaşılır hata mesajı */
    }
}
```

Dikkat

Aynı ürün için eşzamanlı stok düşme gerçek sistemlerde yarış durumuna yol açabilir. Bu console projesinde farklı ürünlerle çalış; thread güvenliğini ileride ele alacağız.

Menü çağırısı

```
case "5":
    await RunOrdersAsync(products);
    break;
```


8. Test, mülakat ve Git

Minimum test listesi

- Geçerli ürün ekleme
- Harfli veya negatif fiyat
- Boş listede listeleme ve rapor
- Var olan ve olmayan ürün arama
- Yetersiz stokla sipariş
- İki siparişin Task.WhenAll ile tamamlanması

Mülakat hafıza notları

- Encapsulation: veriyi kontrollü metotlarla değiştirmek.
- List dinamiktir; Dictionary anahtar-değer; HashSet tekrarsız değer tutar.
- throw hatayı üretir, catch yönetir, finally her durumda çalışır.
- Where filtreler, Select dönüştürür, GroupBy gruplar.
- Task<T> tamamlandığında T tipinde sonuç verir.

Proje ve Git

```
cd "C:\Users\Administrator\.NetEgitim\DotNetIlkHafta"
mkdir Gun7-MiniProje
cd Gun7-MiniProje
dotnet new console
code .

cd "C:\Users\Administrator\.NetEgitim"
git add .
git commit -m "Gün 7 stok ve sipariş mini projesi"
git push
```

1. hafta tamamlanma ölçütü

Projeyi açıklamadan kopyalamak değil; her metodun sorumluluğunu ve kullandığın C# yapısını kendi cümlemlerle anlatabilmek.